Tomasulo limitations

HPS$_m$

C.D.B. limits # inst. that can complete each cycle

⇒ more than one CDB

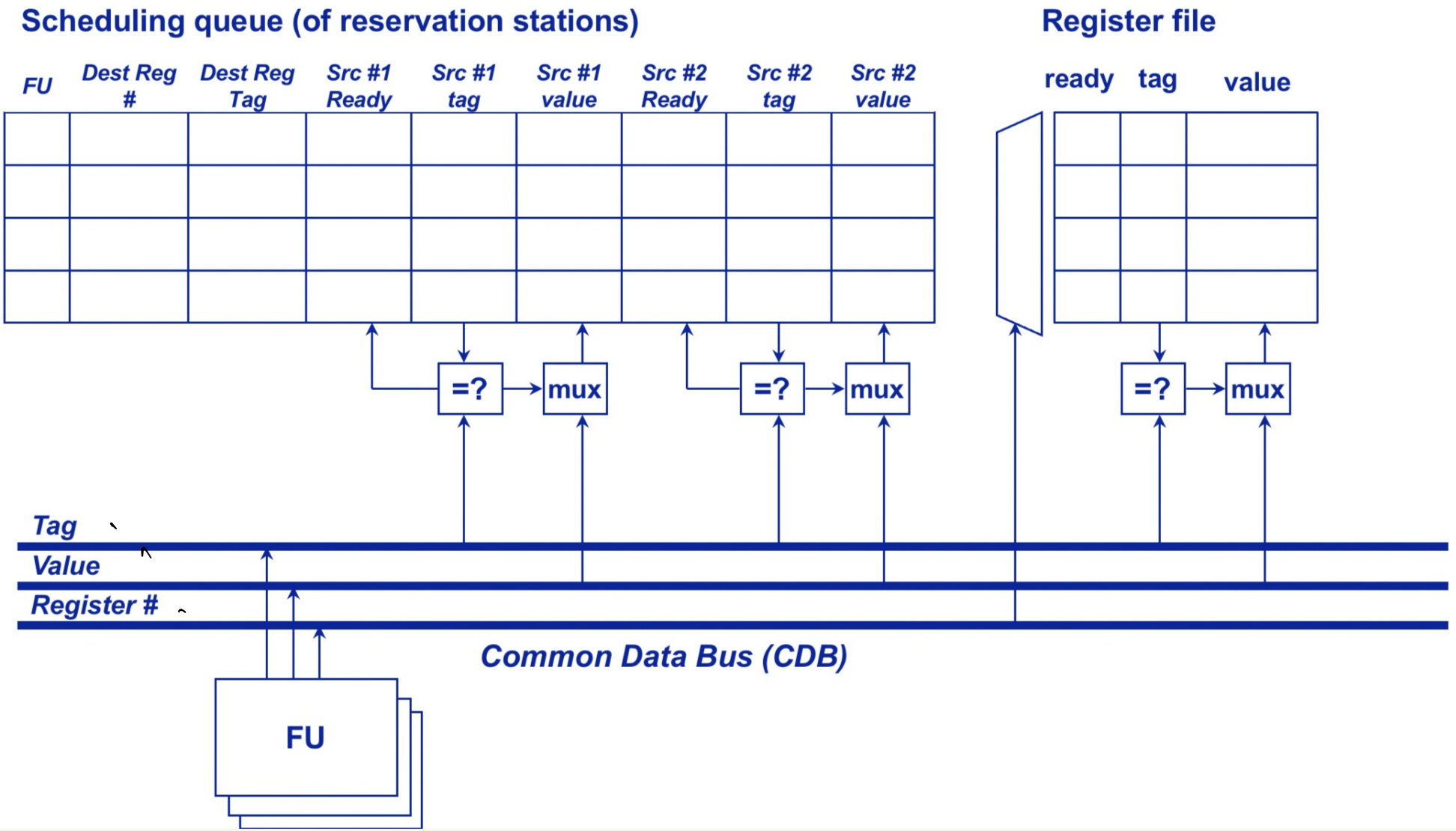
"Result buses"

Best is 1 result bus per FU

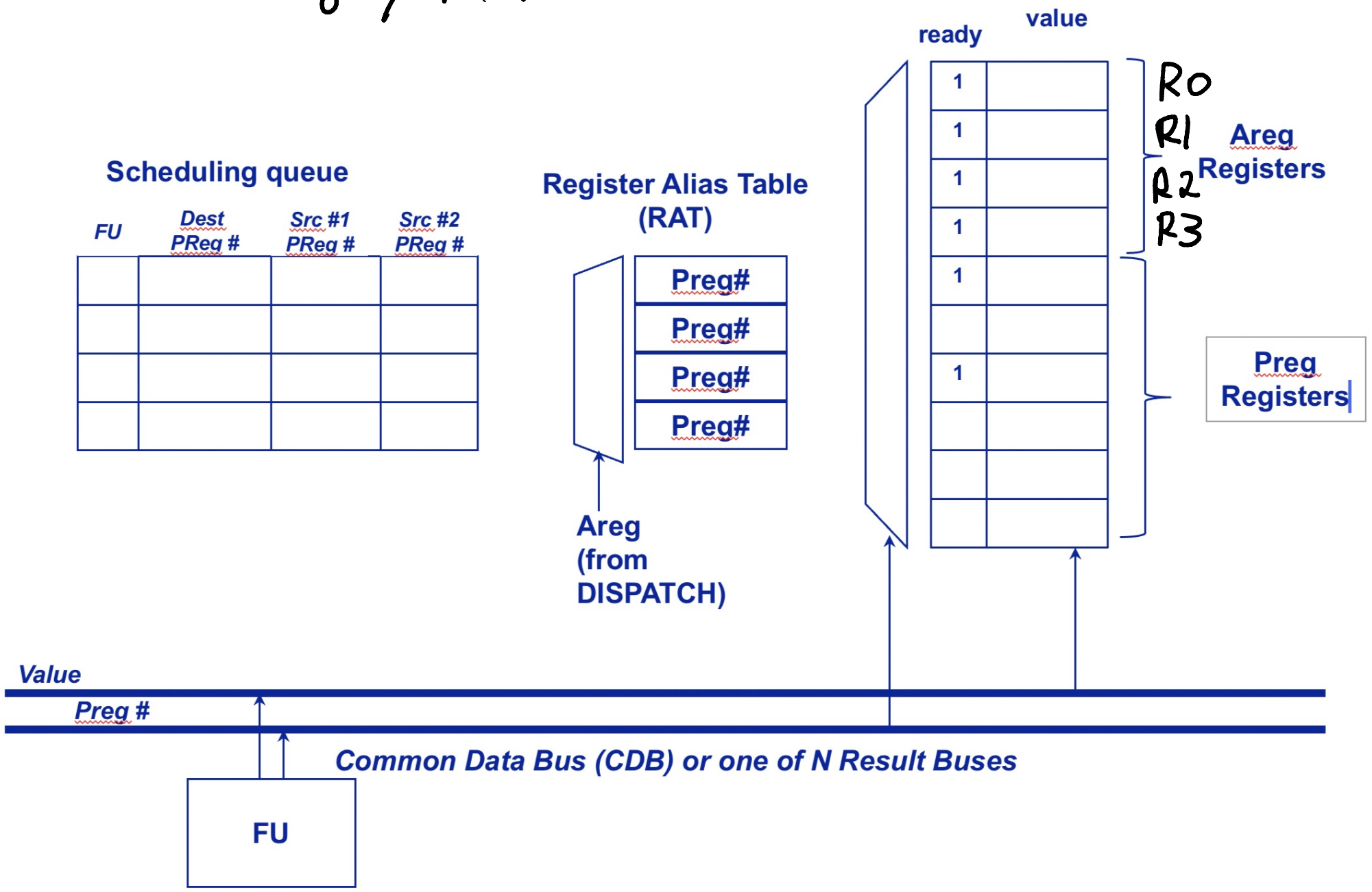
Scheduling queue is complex

Fully associative

Complexity grows with # result buses


Scheduling queue (of reservation stations)
Register file
FU
Dest Reg #
Dest Reg Tag
Src #1 Ready
Src #1 tag
Src #1 value
Src #2 Ready
Src #2 tag
Src #2 value
ready
tag
value
=?
mux
=?
mux
=?
mux
Tag
Value
Register #
Common Data Bus (CDB)
FU

Pregs / RAT


ready
value
1
1
1
1
1
1
1
R0
R1
R2
R3
Areg Registers
Preg Registers
Scheduling queue
FU
Dest PReg #
Src #1 PReg #
Src #2 PReg #
Register Alias Table (RAT)
Preg#
Preg#
Preg#
Preg#
Areg (from DISPATCH)
Value
Preg #
Common Data Bus (CDB) or one of N Result Buses
FU

Tomasulo with Pregs/RAT, part 1

For each cycle of execution…

```
[Dispatch unit ] for all instructions I in DispQ do:
    if (SchedQ is not full AND there are free Pregs) then
        (a) Add I to first free slot of SchedQ  (="RS")      // Call this RS (for "reservation station") below
        (b) Delete I from DispQ                               // Instruction I is now "dispatched"
        (c) RS.FU = I.FU
        (d) for i = 0..1 do
                RS.SRC[i] = RAT[I.srci]
        (e) RAT[I.Dest] = Find a free Preg #   // rename the dest
        (f) RS.Dest_Preg_index = RAT[I.Dest]
        (g) Regs[ RAT[I.Dest] ].Ready = False
    else exit loop // stop dispatching if scheduling queue is full or there are no free PRegs

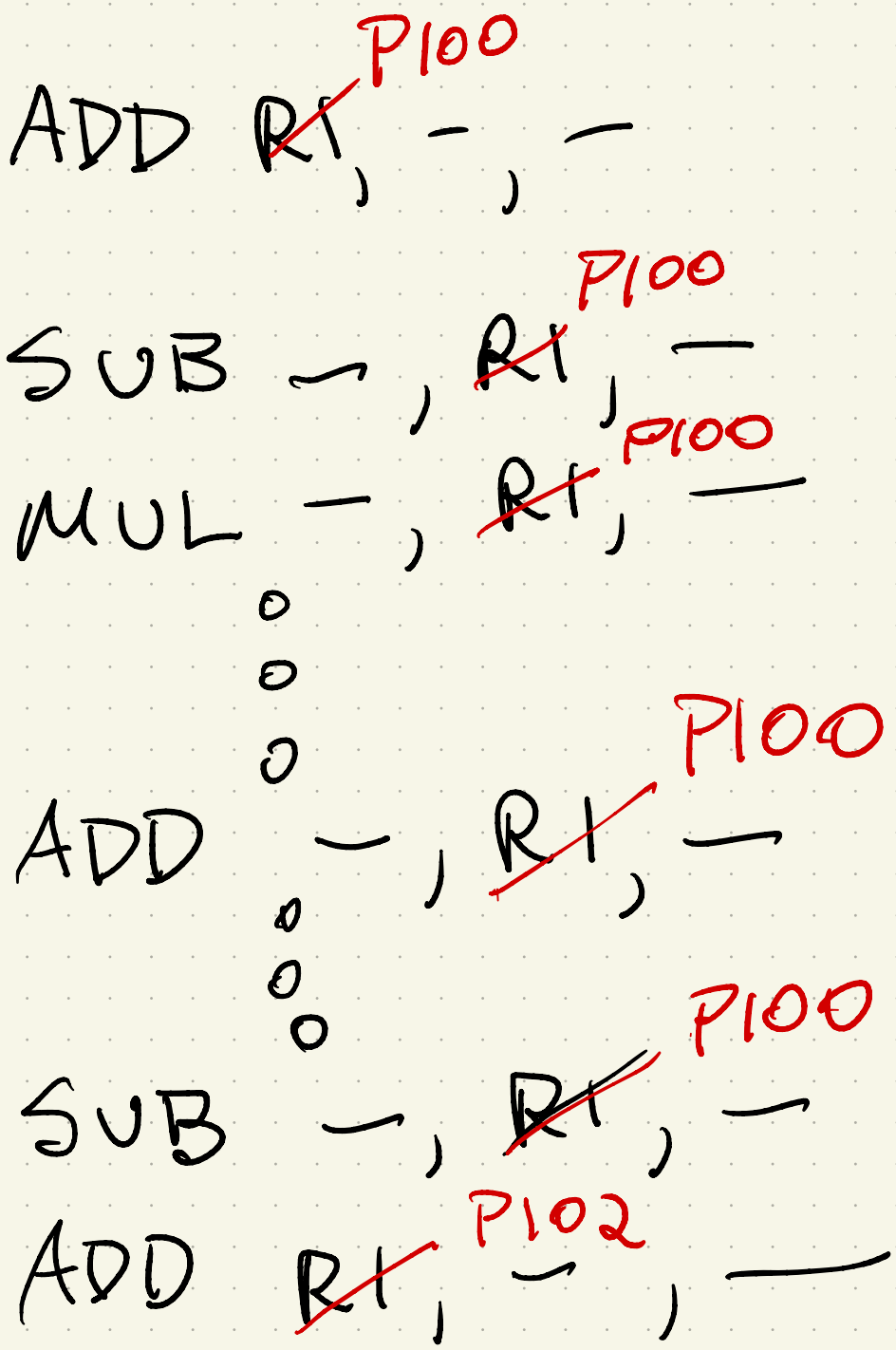

[ Scheduling unit ]
for each RS = entry in SchedQueue do:
    if (Regs[ RS.SRC[0] ].Ready = True AND
        Regs[ RS.SRC[1] ].Ready = True ) // WAKEUP: If all registers are ready...
        and Scoreboard[RS.FU].Busy = False) then
            Scoreboard[RS.FU].Busy = True // ...reserve the FU and issue
            Issue the instruction at RS on FU
```

Tomasulo with Pregs/RAT part 2

```
[ Execution unit, for function unit FU]
    if (FU pipeline advances and now first stage is free) then Scoreboard[FU].Busy = False
[ Execution unit, at completion of instruction I ]
    if (CDB.Busy = False) then                          // Broadcast the instruction results
        (a) CDB.Busy = True
        (b) CDB.Value = I.Value
        (c) CDB.PReg = I.Dest_Preg
        (d) if (I.FU is not pipelined) then
                Scoreboard[I.FU].Busy = False
        (e) Delete I from Sched Queue

[ Register file ]
    if (CDB.Busy = True) then
        (a) Regs[CDB.PReg].Ready = True             // Update the RF contents
        (b) Regs[CDB.PReg].Value = CDB.Value          // update the Preg
        (c) CDB.Busy = False
```

Intel Core 2 microarch



When is it safe to free P100?

```
A  SUB  R1, R4, R5
B  LW   R2, R3(R4)   // 5 cycles, TLB miss
C  SUB  R3, R1, R5
D  SUB  R1, R3, R1
```

TLB miss → OS

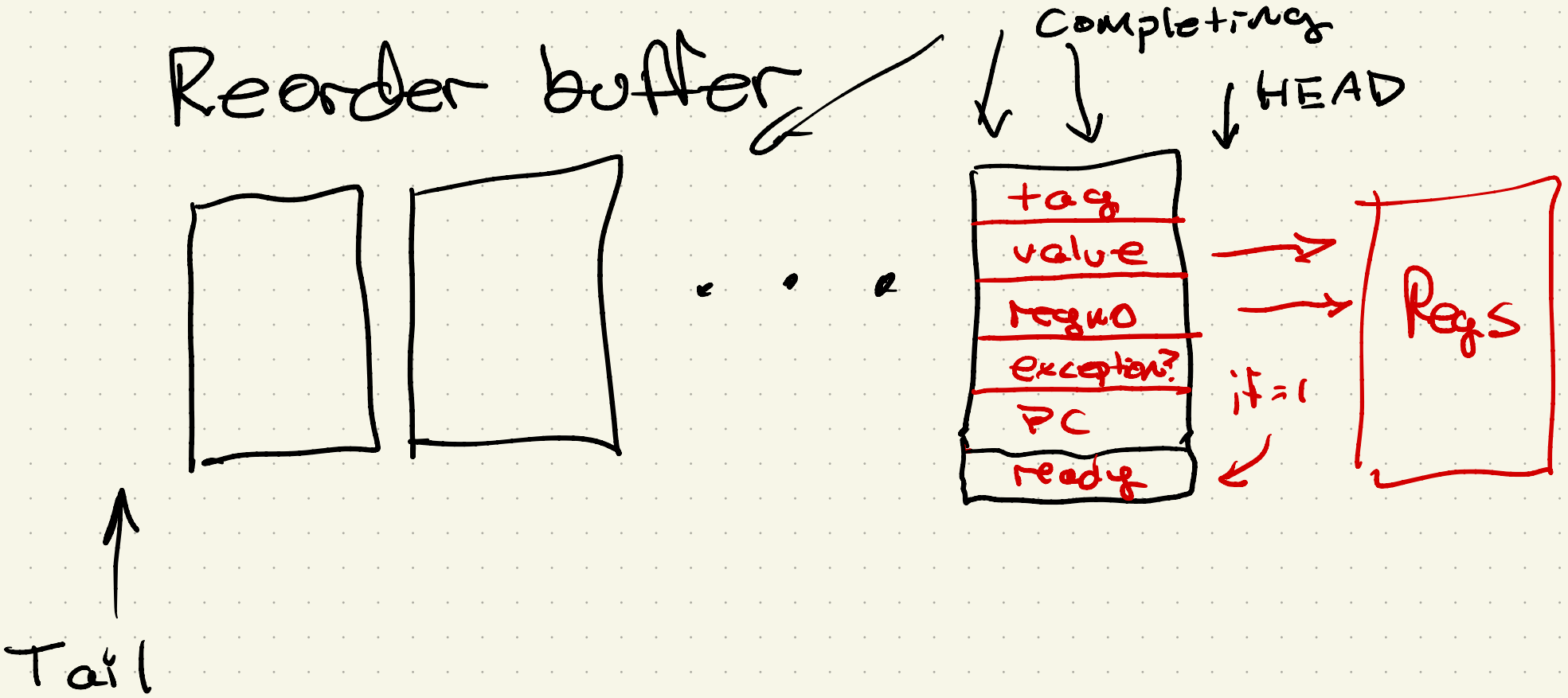
PC value to resume to

```
A  SCH  EX  SU
B  SCH  EX  EX  EX  EX  EX  EX
C  SCH  SCH  EX  SU
D  SCH  SCH  SCH  EX  SU
```

Programmer's view A: R1, B: R2, C: R3, D: R1

Actual: A: R1, C: R3, D: R1, B

Imprecise Interrupt between PC A and D


Reorder buffer
Completing
HEAD
tag
value
regNO
exception?
PC
ready
if=1
Regs
Tail

Tomasulo & ROB w/bypass

```
[Dispatch unit ] for all I in DispatchQueue do:
    if (SchedQueue is not full) then
        (a) Add I to first free slot of SchedQueue  (="RS")        // Call this RS below
        (b) Delete I from DispatchQueue                            // Inst is now "dispatched"
        (c) RS.FU = I.FU
        (d) RS.DestReg = I.Dest
        (e) for i = 0 to Max Source Reg-1
                for j = ROB.tail to ROB.head                       // search for source registers in ROB
                    if (ROB[j].regno = RS.Src[i]) then
                            found_in_rob = True
                            if (ROB[j].Ready = True) then
                                    RS.Src[i].Value = ROB[j].Value
                                    RS.Src[i].Ready = True
                            else
                                    RS.Src[i].Tag = ROB[j].Tag  // Copy the tags from the ROB
                                    RS.Src[i].Ready = False
                            break
                if (found_in_rob = True) then continue  // go to next source reg
                else // no write of reg in ROB, get the value from the register file (because instruction retired long ago!)
                    RS.Src[i].Value = Regs[RS.Src[i]].Value
                    RS.Src[i].Ready = True

        (f) ROB[tail].Tag = unique tag id          // tag (rename) the dest, could just use the ROB index ('tail') as tag#
        (g) ROB[tail].Regno = I.Dest
        (h) ROB[tail].Ready = False
        (i) ROB[tail].Exception = False
        (j) tail++
```

Tomasulo & ROB w/bypass (cont.)

[At instruction completion of instruction I at index 'index' in the ROB]:

(a) ROB[index].Value = I.Value
(b) ROB[index].Ready= True
(c) If *exception for instruction I* then
ROB[index].Exception = true

[At instruction retire, at state update unit]:

if (ROB[head].exception = false) then
Regs[ROB[head].Regno].Value = ROB[head].Value
else /* exception occurred */
(a) resume_pc = ROB[head].pc
(b) tail = head // flush the ROB
(b) *call exception handler*
(c) if *trap*, then pc = resume_pc+4
else /* fault */ pc = resume_pc

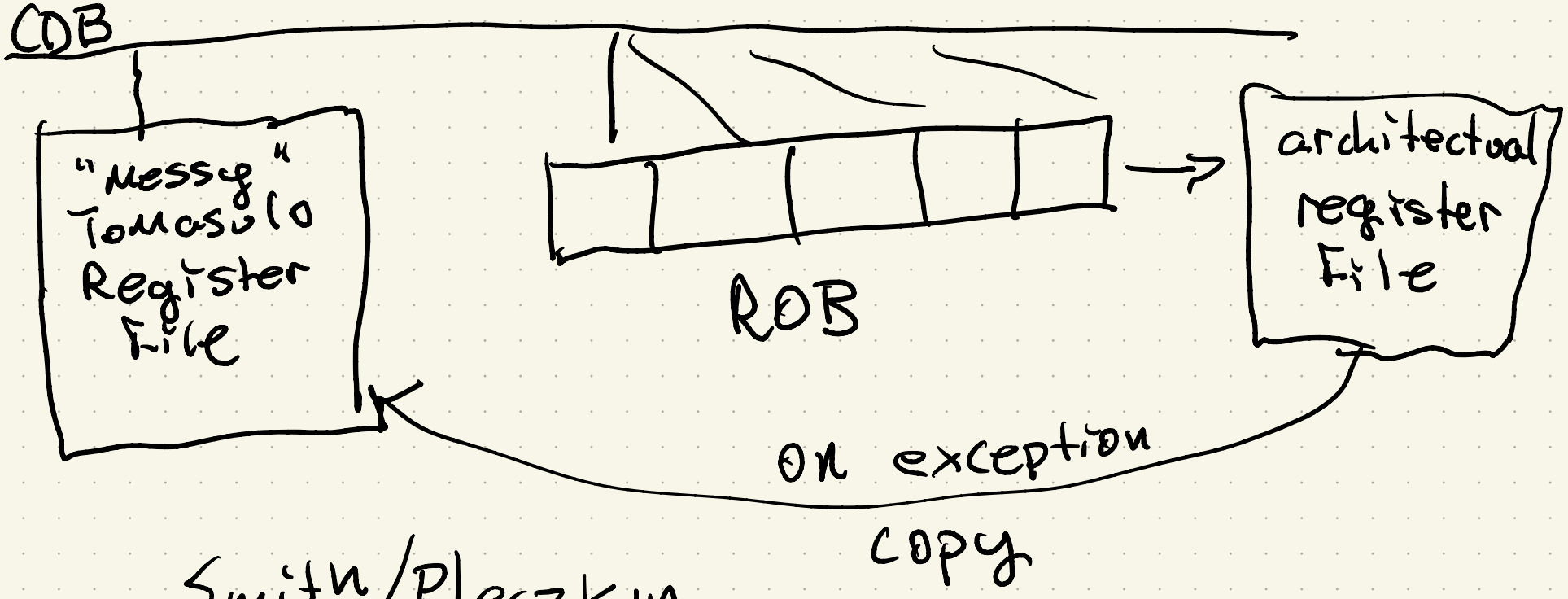
AMD K4 486 - lawsuit

Intel ~~586~~ Pentium

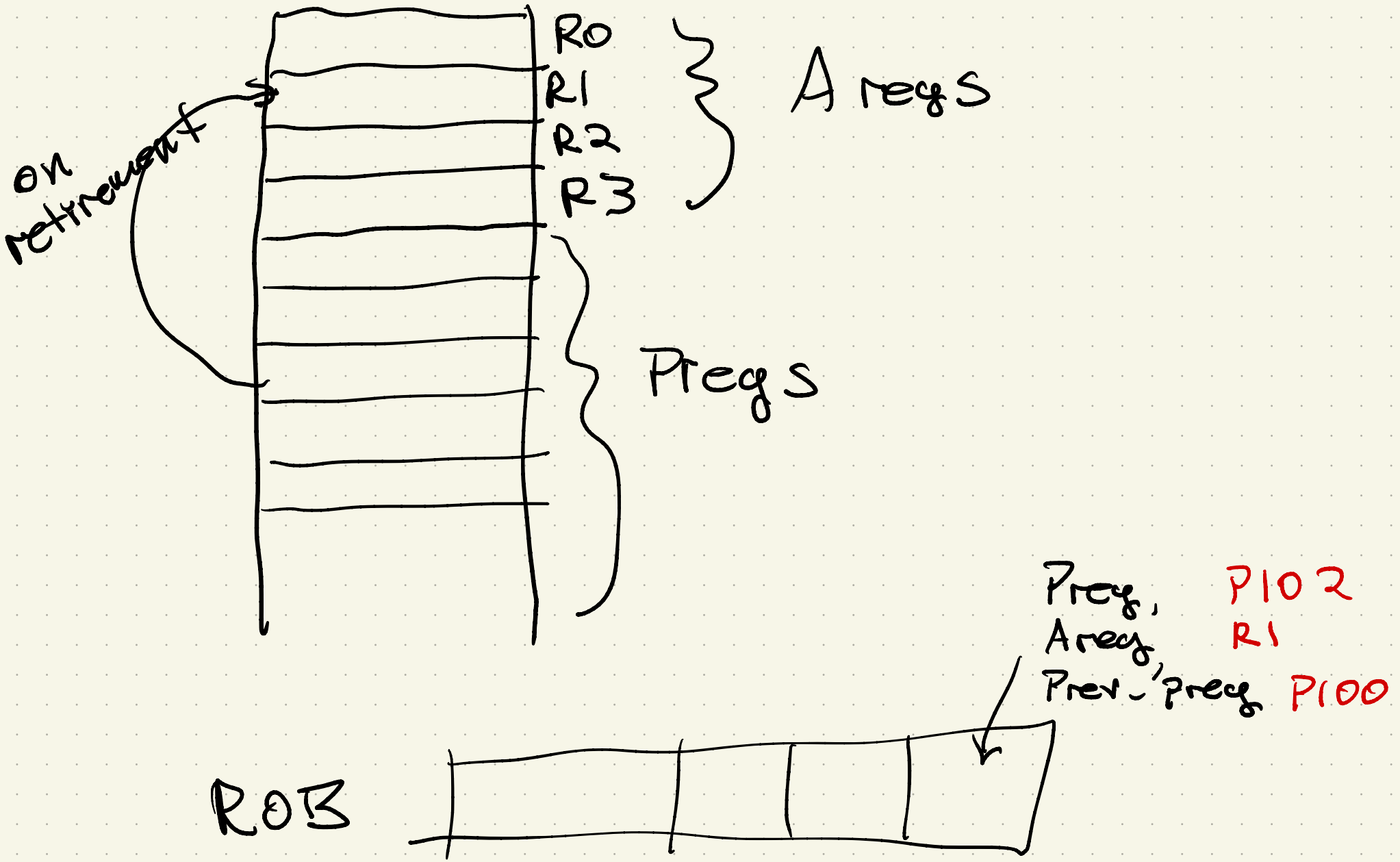
Bought AMD K5 - late, ROB critical path

↳ NextGen - startup in Bay area

ROB w/ Future File



Smith/Pleszkun 1988


R0
R1
R2
R3
Aregs
on retirement
Pregs
Preg, P102
Areg, R1
Prev_Preg P100
ROB

Freeing Pregs:

At Dispatch of I

Allocate slot at tail of ROB

ROB[tail].Prev_preg = RAT[I.Dest]

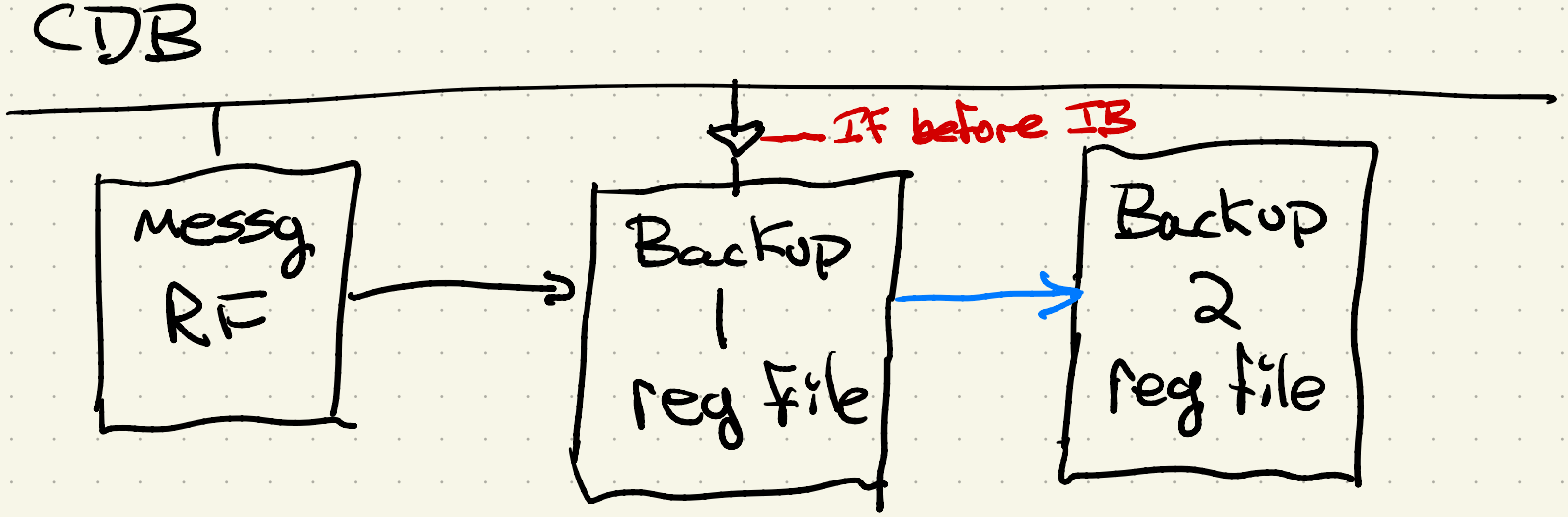
Then rename I.Dest

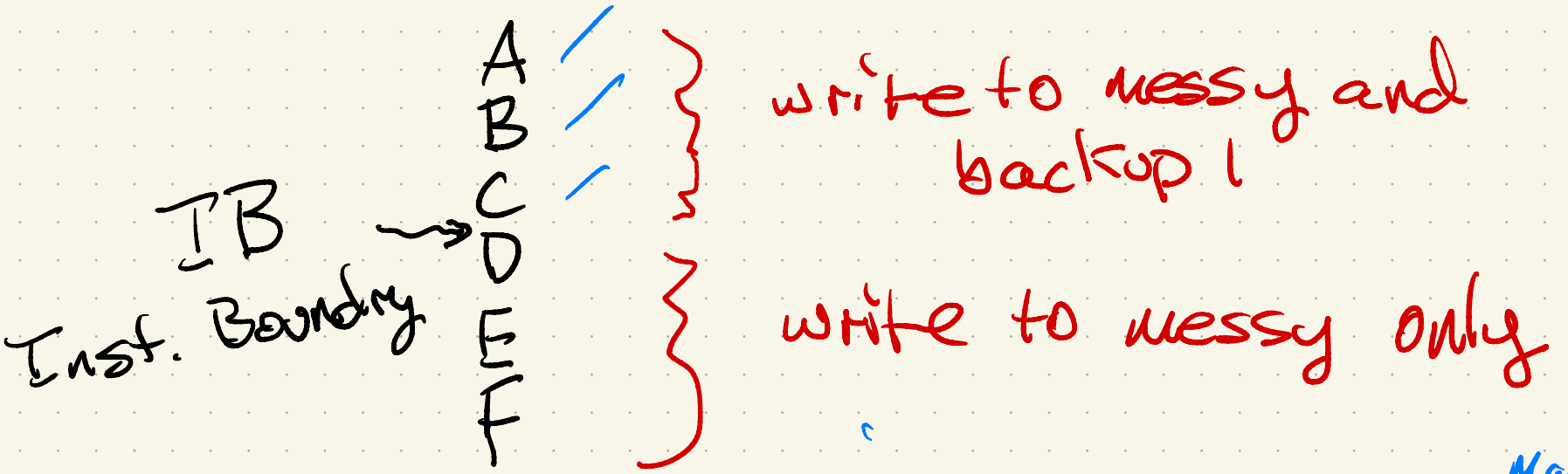
ROB[tail].Preg = Preg # from free list

On instruction retirement:

Free ROB[head].Prev_preg

Checkpoint Repair Patt/Hwu





if all inst before IB complete, make new IB

copy Backup1 to Backup2, Messy to Backup1

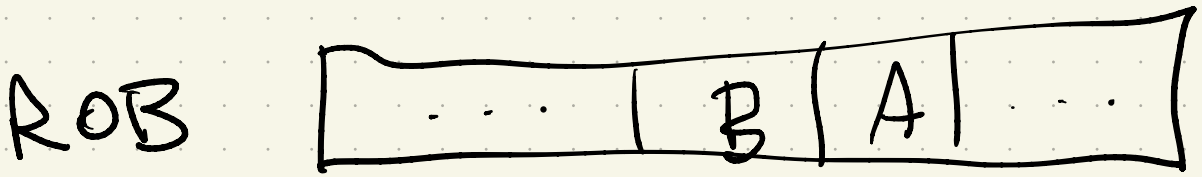
Meltdown

A LW R1, <protected address>

⋮

B LW R2, 0(R1)





1990s — 2000s

P6 ... Pentium 4 ... - Core

IHDL ⟶ Verilog